Semantic tagging

Haystack v1 — Legacy Single-Table Design · built Nov 13, 2026

Haystack v1 — Legacy Single-Table Design

> ⚠ **DEPRECATED — June 2026** Replaced by v2-haystack-current-implementation.md

Purpose

This guide explains exactly how to integrate Project Haystack 4 for T3000 input points, what must be done manually one time, what is automatic, and what data is generated for each point.

Scope of this guide:

- API support and file download support only
- No outbound push workflow
- Input points first (from INPUTS table)

What Haystack Is In This Project

Haystack is a data modeling standard. In this project, integration means:

- Add semantic tags for points such as point, sensor, temp, kind
- Expose standardized APIs to read tagged metadata and history
- Provide file download in Haystack compatible formats

Important:

- You do not manually tag every point row forever
- You define mapping rules once, then code applies them automatically

Existing Source Tables Used

This project already has source tables in webview_t3_device schema:

- DEVICES
- INPUTS
- OUTPUTS
- VARIABLES

For this guide, only INPUTS is used for the end-to-end example.

New Table To Add

Create one new table for normalized Haystack entities.

```sql
CREATE TABLE IF NOT EXISTS HAYSTACK_ENTITY (
    id TEXT PRIMARY KEY,              -- example: dev1001.in0
    kind TEXT NOT NULL,               -- site | equip | point
    dis TEXT,                         -- display name
    tags TEXT NOT NULL,               -- JSON object as string
    serial_number INTEGER,            -- link to DEVICES.SerialNumber
    point_table TEXT,                 -- INPUTS | OUTPUTS | VARIABLES
    point_index TEXT,                 -- Input_Index / Output_Index / Variable_Index
    updated_at INTEGER                -- epoch millis
);

CREATE INDEX IF NOT EXISTS idx_haystack_entity_kind ON HAYSTACK_ENTITY(kind);
CREATE INDEX IF NOT EXISTS idx_haystack_entity_serial ON HAYSTACK_ENTITY(serial_number);
CREATE INDEX IF NOT EXISTS idx_haystack_entity_point_table ON HAYSTACK_ENTITY(point_tabl
```

Manual Work Versus Automatic Work

Manual one-time work:

1. Add migration to create HAYSTACK_ENTITY
2. Implement mapping function from INPUTS row to Haystack tags
3. Implement seeder or upsert job that runs on startup and after sync
4. Add API endpoints and file download endpoint

Automatic recurring work:

1. Read INPUTS rows
2. Compute tags by rule
3. Insert or update HAYSTACK_ENTITY rows
4. Recompute tags on future updates

No manual per-point editing is required in normal operation.

One Input Point End-To-End Example

Source INPUTS row:

- SerialNumber: 1001
- Input_Index: 0
- Label: Zone Temp
- Full_Label: Office 201 Zone Temperature
- fValue: 72.3
- Units: F
- Digital_Analog: 1

Generated Haystack entity:

- id: dev1001.in0
- kind: point
- dis: Office 201 Zone Temperature
- serial_number: 1001
- point_table: INPUTS
- point_index: 0
- tags JSON:

```JSON
{
  "point": "M",
  "sensor": "M",
  "temp": "M",
  "his": "M",
  "kind": "Number",
  "unit": "F",
  "curVal": 72.3,
  "equipRef": "dev1001"
}
```

Meaning:

- point M: this entity is a point
- sensor M: this point is a sensor
- temp M: this point is temperature related
- kind Number: value is numeric

Mapping Rules For INPUTS

Minimum mapping rules:

1. Base tags:

- point = M
- sensor = M
- his = M

2. Kind mapping from Digital_Analog:

- Digital_Analog = 1 -> kind = Number
- Digital_Analog = 0 -> kind = Bool

3. Semantic mapping from Units:

- F or C -> temp = M
- % -> humidity = M
- ppm -> co2 = M
- V -> volt = M and elec = M
- A -> current = M and elec = M

4. Value mapping:

- unit = Units
- curVal = parsed numeric value from fValue

5. Relationship mapping:

- equipRef = dev + SerialNumber

Recommended Seeder Behavior

Run seeder at two times:

1. Backend startup
2. After each successful device or point sync

Upsert strategy:

- If id does not exist in HAYSTACK_ENTITY, insert row
- If id exists, update dis, tags, updated_at

This ensures tag changes are automatic if metadata changes.

Rust Pseudocode For INPUTS Upsert

RUST

```rust
for input in load_all_inputs(db).await? {
    let id = format!("dev{}.in{}", input.serial_number, input.input_index);
    let dis = pick_display_name(input.full_label.as_deref(), input.label.as_deref());

    let mut tags = serde_json::Map::new();
    tags.insert("point".into(), serde_json::json!("M"));
    tags.insert("sensor".into(), serde_json::json!("M"));
    tags.insert("his".into(), serde_json::json!("M"));

    if input.digital_analog == "1" {
        tags.insert("kind".into(), serde_json::json!("Number"));
    } else {
        tags.insert("kind".into(), serde_json::json!("Bool"));
    }

    match input.units.as_deref().unwrap_or("") {
        "F" | "C" => { tags.insert("temp".into(), serde_json::json!("M")); }
        "%" => { tags.insert("humidity".into(), serde_json::json!("M")); }
        "ppm" => { tags.insert("co2".into(), serde_json::json!("M")); }
        "V" => {
            tags.insert("volt".into(), serde_json::json!("M"));
            tags.insert("elec".into(), serde_json::json!("M"));
        }
        "A" => {
            tags.insert("current".into(), serde_json::json!("M"));
            tags.insert("elec".into(), serde_json::json!("M"));
        }
        _ => {}
    }

    if let Some(unit) = input.units.as_deref() {
        tags.insert("unit".into(), serde_json::json!(unit));
    }
    if let Ok(v) = input.fvalue.parse::<f64>() {
        tags.insert("curVal".into(), serde_json::json!(v));
    }

    tags.insert("equipRef".into(), serde_json::json!(format!("dev{}", input.serial_numbe
```

```
    upsert_haystack_entity(
        db,
        &id,
        "point",
        &dis,
        &serde_json::Value::Object(tags).to_string(),
        input.serial_number,
        "INPUTS",
        &input.input_index,
    ).await?;
}
```

Required API Endpoints

Implement only these endpoints for current scope.

1. POST /api/haystack/about

- Returns server metadata and supported Haystack version

2. POST /api/haystack/read

- Filter entities by tags
- Returns Haystack grid style response

3. POST /api/haystack/hisRead

- Input: entity id and time range
- Reads trend history from existing trendlog data
- Returns ts and val rows in Haystack grid style

4. GET /api/haystack/download

- Query parameters: format, filter, range, ids
- Streams file attachment
- Recommended formats: json and csv initially

Suggested Download Behavior

For MVP:

- format=json: download Haystack grid JSON
- format=csv: download flattened rows

Response headers:

- Content-Type based on format
- Content-Disposition: attachment; filename="haystack_export_."

Validation Checklist

1. Migration applied and HAYSTACK_ENTITY exists
2. Seeder inserted rows for INPUTS points
3. One known point returns expected tags (point, sensor, kind)
4. If Digital_Analog changes, kind updates on next upsert
5. read endpoint returns filtered point
6. hisRead endpoint returns time series for that point
7. download endpoint saves file successfully

Common Mistakes To Avoid

1. Manual per-point tagging in UI

- Do not do this except special override cases

2. Storing tags in many separate columns

- Keep tags as JSON for flexibility

3. One-time seed only

- Must also run upsert after sync to keep data current

4. Hardcoding only one unit format

- Support both plain and symbol variants where possible

Next Increment After INPUTS

After INPUTS is stable, repeat the same pattern for:

- OUTPUTS
- VARIABLES

Use id patterns:

- dev{SerialNumber}.out{Output_Index}
- dev{SerialNumber}.var{Variable_Index}

Then expose them through the same read and download endpoints.